Statistical and Machine Learning SS26

Exercise Sheet 4 - Solutions

Setup

```
#install.packages("faraway") #remove # to install package
library(faraway) #load package

data("fat") # load dataset
str(fat) # 252 observations, 19 variables (18 predictors + 1 response)
```

```
## 'data.frame':    252 obs. of  18 variables:
##  $ brozek : num  12.6 6.9 24.6 10.9 27.8 20.6 19 12.8 5.1 12 ...
##  $ siri   : num  12.3 6.1 25.3 10.4 28.7 20.9 19.2 12.4 4.1 11.7 ...
##  $ density: num  1.07 1.09 1.04 1.08 1.03 ...
##  $ age    : int  23 22 22 26 24 24 26 25 25 23 ...
##  $ weight : num  154 173 154 185 184 ...
##  $ height : num  67.8 72.2 66.2 72.2 71.2 ...
##  $ adipos : num  23.7 23.4 24.7 24.9 25.6 26.5 26.2 23.6 24.6 25.8 ...
##  $ free   : num  135 161 116 165 133 ...
##  $ neck   : num  36.2 38.5 34 37.4 34.4 39 36.4 37.8 38.1 42.1 ...
##  $ chest  : num  93.1 93.6 95.8 101.8 97.3 ...
##  $ abdom  : num  85.2 83 87.9 86.4 100 94.4 90.7 88.5 82.5 88.6 ...
##  $ hip    : num  94.5 98.7 99.2 101.2 101.9 ...
##  $ thigh  : num  59 58.7 59.6 60.1 63.2 66 58.4 60 62.9 63.1 ...
##  $ knee   : num  37.3 37.3 38.9 37.3 42.2 42 38.3 39.4 38.3 41.7 ...
##  $ ankle  : num  21.9 23.4 24 22.8 24 25.6 22.9 23.2 23.8 25 ...
##  $ biceps : num  32 30.5 28.8 32.4 32.2 35.7 31.9 30.5 35.9 35.6 ...
##  $ forearm: num  27.4 28.9 25.2 29.4 27.7 30.6 27.8 29 31.1 30 ...
##  $ wrist  : num  17.1 18.2 16.6 18.2 17.7 18.8 17.7 18.8 18.2 19.2 ...
```

```
?fat # opens help page for variable definitions
set.seed(1234)
```

---

1) Split data into train and test set

```
train_idx <- sample(1:nrow(fat), ceiling(0.7*nrow(fat)), replace = FALSE)
# samples 70% of row indices randomly and stores them as vector train_idx
# replace_false ensures no index is sampled multiple times

test_idx <- setdiff(1:nrow(fat), train_idx)
```

```r
# stores all indices from 1:252 that are NOT in train_idx

fat_train <- fat[train_idx,]
# selects all rows whose index is contained in train_idx. keeps all columns
fat_test <- fat[test_idx,]
# same thing but for text set
```

---

2) OLS

```r
fat.lm <- lm(brozek ~ ., data = fat_train)
#fit OLS model with all predictors on training set
summary(fat.lm)
```

```
##
## Call:
## lm(formula = brozek ~ ., data = fat_train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.03547 -0.04872  0.01199  0.05235  1.40500
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.256e+01  4.596e+00   2.732  0.00701 **
## siri         8.894e-01  1.384e-02  64.252  < 2e-16 ***
## density     -1.022e+01  4.087e+00  -2.499  0.01346 *
## age          2.261e-04  1.645e-03   0.137  0.89084
## weight       7.637e-03  5.726e-03   1.334  0.18422
## height       9.541e-05  4.781e-03   0.020  0.98410
## adipos      -1.823e-02  1.631e-02  -1.118  0.26529
## free        -8.206e-03  6.406e-03  -1.281  0.20209
## neck         3.301e-03  1.207e-02   0.274  0.78474
## chest        3.367e-03  5.474e-03   0.615  0.53933
## abdom        4.109e-03  5.866e-03   0.701  0.48460
## hip         -6.699e-03  7.572e-03  -0.885  0.37762
## thigh        2.075e-02  7.520e-03   2.759  0.00648 **
## knee        -2.577e-02  1.235e-02  -2.087  0.03849 *
## ankle       -5.548e-03  1.321e-02  -0.420  0.67507
## biceps      -1.495e-02  9.079e-03  -1.646  0.10167
## forearm      1.549e-02  9.504e-03   1.630  0.10515
## wrist        7.461e-03  2.719e-02   0.274  0.78415
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1785 on 159 degrees of freedom
## Multiple R-squared:  0.9995, Adjusted R-squared:  0.9994
## F-statistic: 1.785e+04 on 17 and 159 DF,  p-value: < 2.2e-16
```

```r
#MSE: how far away are our model estimates
fat.lm.pred <- predict(fat.lm, fat_test) #vector of predicted brozek values for test set
MSE.lm <- mean((fat.lm.pred - fat_test$brozek)^2) #compute test MSE
# (predicted - actual brozek values, squared)
MSE.lm
```

```
## [1] 0.02724344
```

c) No, only 3 out of 18 predictors are significant at the 5% level: `siri`, `density`, `thigh` and `knee`. We need to be careful here! Because `siri` and `brozek` are highly correlated and both are calculated by `density`. All other physical measurements (like `neck` or `hip`) will likely appear insignificant because their information is already "captured" by the `density` variable.

d) Multicollinearity: Physical measurements are highly correlated with each other. In OLS, this leads to high variance in coefficient estimates. This instability is a primary motivation for Regularization (Ridge and Lasso), which stabilizes the estimates by adding a penalty term.

---

3) Lasso

```r
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```r
lasso.fat <- glmnet(x = as.matrix(fat_train[,-1]), y = fat_train$brozek, alpha = 1, lambda = 0.03)
# glmnet requires the predictors to be in a numeric matrix! we convert training predictors
# to a matrix and remove the response variable (column 1, 'brozek').
# y: response variable, value we want to predict
#alpha 1: The penalty term in the regression equation is written as a combination of
# the Lasso and Ridge norms. If we set alpha to 1, the first part of the equation (Ridge)
# becomes zero, only the second part (Lasso) remains

coef(lasso.fat) # inspect coefficients
```

```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##                         s0
## (Intercept)  7.231032e+00
## siri         9.118219e-01
## density     -5.554375e+00
## age          .
## weight       .
## height       .
## adipos       .
## free         .
## neck         .
## chest        .
## abdom        5.197853e-05
```

```
## hip          .
## thigh        1.657077e-03
## knee         .
## ankle        .
## biceps       .
## forearm      .
## wrist        .
```

---

4) Ridge

```
ridge.fat <- glmnet(x = as.matrix(fat_train[,-1]), y = fat_train$brozek, alpha = 0, lambda = 0.03)
# same thing but alpha= 0 -> Ridge part of equation remains
coef(ridge.fat)
```

```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##                         s0
## (Intercept)  48.218147219
## siri          0.751412097
## density     -43.671775230
## age           0.001906887
## weight        0.029964343
## height        0.003174876
## adipos       -0.044145062
## free         -0.041427802
## neck          0.011432397
## chest         0.011729424
## abdom         0.008752974
## hip           0.002292739
## thigh         0.025411003
## knee         -0.019665004
## ankle        -0.004446556
## biceps       -0.007680105
## forearm       0.022326740
## wrist         0.007440406
```

---

5) Comparision Ridge - Lasso

```
#compare 'by hand' or:
# install.packages("modelsummary")
library("modelsummary")
modelsummary(list("LASSO" = lasso.fat, "Ridge" = ridge.fat))
```

We can see that Lasso has removed many variables, while Ridge kept even 'unimportant' variables.

Coefficients indicate direction and magnitude of the effect. **Important**: these values have been 'shrunk' by the penalty parameter $\lambda \rightarrow$ they do not represent the unbiased effects typically seen in OLS.

L: for each unit that density increases, the brozek value decreases by approximately 5.55 percentage points.

R: For each additional pound bodyweight, the body fat percentage (bronzek) increases by approx. 0.03 percentage points

Lasso * Uses the absolute value of the coefficients * Sparsity: it performs variable selection: it sets many of them exactly to zero. This is great for interpretation because it tells you exactly which variables are "useless" for prediction. * Diamond shape with sharp corners on the axes (where one coefficient is zero). When the model tries to find the best fit, it is very likely to "hit" one of those sharp corners. * Best used when the model includes many "junk" variables * Handles correlated variables by picking one and dropping the other * Interpretation of R output: dot means the coefficient was shrunk exactly to zero

Ridge * Uses the square of the coefficients * Shrinkage: it shrinks all coefficients toward zero, but they almost never reach exactly zero. * You keep all your predictors in the model, but they have less "influence." * Circle shape: the model is very unlikely to hit the axis exactly at zero; it will just get closer and closer. * Best used when many variables are useful * Shrinks correlated variables together

---

6) Select λ by CV

a) Choosing the "wrong" λ can lead to two major problems:

Too small λ: You don't penalize enough, leading to overfitting (high variance).

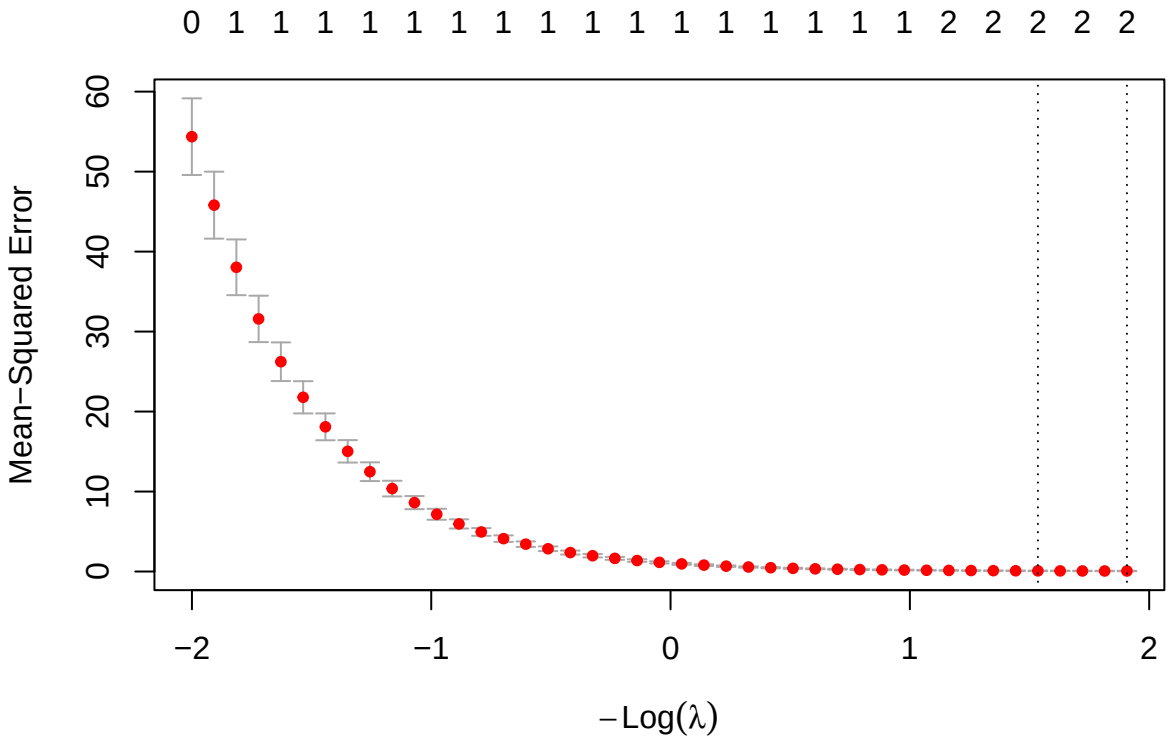
Too large λ: You penalize too much, leading to underfitting (high bias).

$\rightarrow$ Cross-Validation to find λ that minimizes prediction error

b) & c)

```
#LASSO:
lasso.fat.cv <- cv.glmnet(x = as.matrix(fat_train[,-1]), y = fat_train$brozek, alpha = 1)
#again, we leave out the first column (brozek), define the target variable and set alpha to 1

plot(lasso.fat.cv)
```



cv.glmnet() runs the model many times over a grid of different λ values.

10-Fold Cross-Validation: By default, it splits your training data into 10 parts (folds). It trains on 9 and tests on 1, rotating until every fold has been used for testing.

The Goal: It calculates the average error (MSE) for every λ to find which one performs best on "unseen" data.

Plot: We use the log scale because λ changes exponentially. Without the log transformation, the plot would be unreadable, as the transition from a complex model to a simple one happens very quickly at small values of λ.

- Moving left means a smaller penalty (approaching OLS).
- Moving right means a stronger penalty (shrinking more coefficients to zero) **it should rebounce** -> stopping criteria in plot?

Y-axis (Mean-Squared Error): Aaverage prediction error across your 10 folds, lower is better

Dotted vertical Lines: "answers" the CV process found

Left Dotted Line ($\lambda.min$): The λ that produced the absolute lowest MSE. The best performing model for prediction

Right Dotted Line ($\lambda.1se$): This is the "One Standard Error" rule. It is a slightly more restrictive (higher λ) model. We use this if we want a simpler model that performs almost as well as the minimum

Goal: We want to be in that flat valley between the two dotted lines where the error is minimized

Numbers at the Top: these integers (0, 1, 1, 1, 2, 2...) indicate the number of non-zero coefficients in the model at that specific λ.

Interpretation: Notice that as we move to the right (stronger penalty), the number of variables in the model drops. In your plot, the "optimal" models only use 2 variables (likely siri and density from your previous table).

Red Dots: The average MSE for that λ and their variation (standard error) of the MSE across the 10 folds. If these bars are huge, it means the model's performance is very inconsistent across different subsets of data.

Error so high on the far left, because the penalty is too small. The model is over-complex and fitting the "noise" in the training data (high variance).

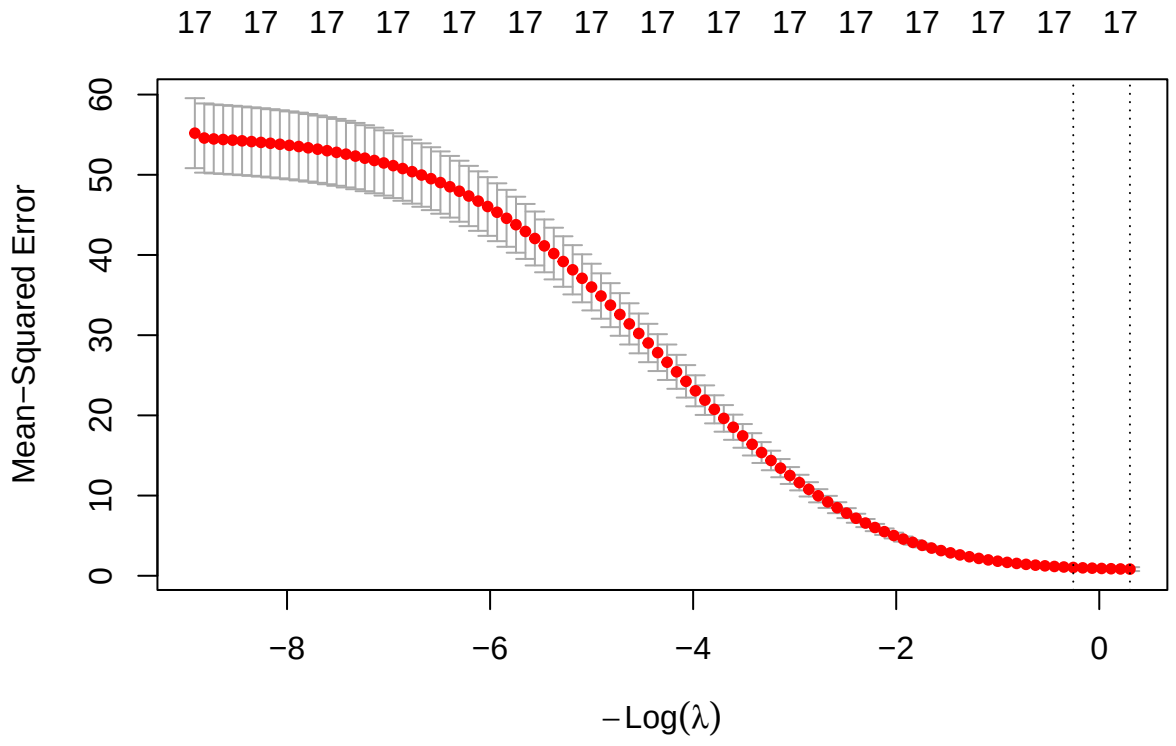
```
lasso.fat.cv$lambda.min #gives optimal lambda values
```

```
## [1] 0.1485119
```

Same procedure for Ridge:

```
ridge.fat.cv <- cv.glmnet(x = as.matrix(fat_train[,-1]), y = fat_train$brozek, alpha = 0)

plot(ridge.fat.cv)
```




```
ridge.fat.cv$lambda.min
```

```
## [1] 0.7391471
```

7)

```
coef(lasso.fat.cv, s ="lambda.min")
```

```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##             lambda.min
## (Intercept)  4.0929044
## siri         0.9047184
## density     -2.3533679
## age          .
## weight       .
## height       .
## adipos       .
## free         .
## neck         .
## chest        .
## abdom        .
## hip          .
## thigh        .
## knee         .
## ankle        .
## biceps       .
## forearm      .
## wrist        .
```

Using the optimal λ from cross-validation, the Lasso model identifies the predictors that are most useful for predicting body fat while forcing the coefficients of less relevant variables to zero.

The Lasso model selects only two predictors: `siri` and `density`. All others (age, weight, height, and all 10 circumference measurements) have been excluded from the model, indicated by the dots. Makes sense because once the model has access to `density` or `siri`, physical measurements provide no additional information. Lasso identifies this redundancy and removes the physical measurements to create the simplest possible model.

8)

a)

```
fat.lm$coefficients #coeffs of OLS
```

```
##   (Intercept)          siri       density           age        weight
##  1.255682e+01  8.894359e-01 -1.021544e+01  2.261468e-04  7.637096e-03
##        height        adipos          free          neck         chest
##  9.540921e-05 -1.822998e-02 -8.205612e-03  3.301329e-03  3.367499e-03
##         abdom           hip         thigh          knee         ankle
##  4.109438e-03 -6.699097e-03  2.074729e-02 -2.577105e-02 -5.548143e-03
##        biceps       forearm         wrist
## -1.494773e-02  1.548856e-02  7.461239e-03
```

```r
coef(ridge.fat.cv) #coeffs of ridge
```

```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##                 lambda.1se
## (Intercept)  1.266465e+02
## siri         3.507815e-01
## density     -1.270324e+02
## age          1.785087e-02
## weight       1.920191e-02
## height       2.689585e-02
## adipos       5.624952e-02
## free        -7.710731e-02
## neck         7.395872e-03
## chest        4.052761e-02
## abdom        7.311806e-02
## hip          5.544971e-02
## thigh        5.082506e-02
## knee         5.385198e-02
## ankle        1.831530e-02
## biceps       4.969662e-02
## forearm      5.262524e-02
## wrist       -1.142735e-01
```

```r
coef(lasso.fat.cv, s = "lambda.min") #coeffs of lasso with optimal lambda
```

```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##             lambda.min
## (Intercept)  4.0929044
## siri         0.9047184
## density     -2.3533679
## age          .
## weight       .
## height       .
## adipos       .
## free         .
## neck         .
## chest        .
## abdom        .
## hip          .
## thigh        .
## knee         .
## ankle        .
## biceps       .
## forearm      .
## wrist        .
```

```r
#MSE OLS
fat.lm.pred <- predict(fat.lm, fat_test)

MSE.lm <- mean((fat.lm.pred - fat_test$brozek)^2)
```

```r
#MSE Ridge
fat.ridge.pred <- predict(ridge.fat.cv, as.matrix(fat_test[,-1]), s = "lambda.min")

MSE.ridge <- mean((fat.ridge.pred - fat_test$brozek)^2)
```

```r
#MSE Lasso
fat.lasso.pred <- predict(lasso.fat.cv, as.matrix(fat_test[,-1]), s = "lambda.min")

MSE.lasso <- mean((fat.lasso.pred - fat_test$brozek)^2)
```

```r
#compare:
MSE_comparison <- c(OLS = MSE.lm, Ridge = MSE.ridge, Lasso = MSE.lasso)
MSE_comparison
```

```
##        OLS      Ridge      Lasso
## 0.02724344 0.44427554 0.03392651
```

b) The MSE results show a clear ranking in predictive accuracy for this specific test set:

OLS (0.0272): Achieved the best performance. This is because the dataset contains predictors (siri and density) that have a near-perfect mathematical relationship with the target variable brozek.

Lasso (0.0339): Performed slightly worse than OLS. While it correctly identified the most important variables, the intrinsic shrinkage bias (pulling coefficients toward zero) resulted in a minor loss of accuracy.

Ridge (0.4443): Had the highest error. Since Ridge cannot set coefficients to zero, it forced all redundant and "noisy" variables to remain in the model, which was inefficient for this specific data structure.

But! If we re-run the code without a seed, your data is being split into different train and test sets each time. This creates sampling variability and therefore different MSEs. I ran it approx. 10 times and either Lasso or OLS always had the smallest MSE, Ridge had the highest error.

c) While OLS has the lowest bias, its high variance usually leads to a high total MSE on new data. By using Ridge or Lasso, we accept a little bit of Bias (an intentional error) in exchange for a massive reduction in Variance → lower overall MSE **But!** Not always :)